

Removes project

```
CALL gds.graph.drop('ontoloasxGraph')
YIELD graphName;
```

Creates project

```
CALL gds.graph.project(
  'ontoloasxGraph',

  [
    'Study',
    'Solution',
    'SolutionElement',
    'Context',
    'Limitation'
  ],

  {
    PROPOSES: {
      orientation: 'UNDIRECTED'
    },
    HAS_ELEMENT: {
      orientation: 'UNDIRECTED'
    },
    APPLIES_TO: {
      orientation: 'UNDIRECTED'
    },
    HAS_LIMITATION: {
      orientation: 'UNDIRECTED'
    }
  }
)
YIELD
  graphName,
  nodeCount,
  relationshipCount;
```

Executes PageRank

```
CALL gds.pageRank.write(
  'ontoloasxGraph',
  {
    writeProperty: 'pageRank',
    maxIterations: 20,
    dampingFactor: 0.85
  }
)
YIELD
  nodePropertiesWritten,
  ranIterations;
```

Queries PageRank's results

```
MATCH (n)
WHERE n.pageRank IS NOT NULL
RETURN
  coalesce(n.id,n.title,n.description,n.number) AS Concept,
  labels(n) AS Class,
  round(n.pageRank,6) AS Score
ORDER BY Score DESC;
```

Executes Betweenness Centrality

```
CALL gds.betweenness.write(  
    'ontoloasxGraph',  
    {  
        writeProperty: 'betweenness'  
    }  
)  
YIELD  
    nodePropertiesWritten,  
    centralityDistribution;
```

Queries top 10 central contexts

```
MATCH (c:Context)  
WHERE c.betweenness IS NOT NULL  
RETURN  
coalesce(c.id,c.title,c.description) AS Context,  
round(c.betweenness,2) AS Betweenness  
ORDER BY c.betweenness DESC  
LIMIT 10;
```

Queries top 10 central studies

```
MATCH (c:Study)  
WHERE c.betweenness IS NOT NULL  
RETURN  
coalesce(c.id,c.title,c.description) AS Study,  
round(c.betweenness,2) AS Betweenness  
ORDER BY c.betweenness DESC  
LIMIT 10;
```

Removes project

```
CALL gds.graph.drop('ontoloasxGraph', false)  
YIELD graphName;
```

Creates Louvain Modularity community evidence project

```
CALL gds.graph.project(  
    'ontoloasxGraph',  
  
    [  
        'Solution',  
        'Context',  
        'SolutionElement'  
    ],  
  
    {  
        APPLIES_TO: {  
            orientation: 'UNDIRECTED'  
        },  
        HAS_ELEMENT: {  
            orientation: 'UNDIRECTED'  
        }  
    }  
)  
YIELD  
    graphName,  
    nodeCount,
```

```
relationshipCount;
```

Executes Louvain Modularity (Context, Solution, SolutionElement)

```
CALL gds.louvain.write(  
  'ontoloasxGraph',  
  {  
    writeProperty: 'louvainCommunity'  
  }  
)  
YIELD  
  communityCount,  
  modularity,  
  modularities;
```

Show communities identified

```
MATCH (n)  
WHERE n.louvainCommunity IS NOT NULL  
  AND (n:Solution OR n:Context OR n:SolutionElement)  
RETURN  
  n.louvainCommunity AS Community,  
  count(n) AS Nodes,  
  collect(coalesce(n.id,n.title,n.description))[0..20] AS  
SampleEntities  
ORDER BY Nodes DESC;
```

Summarizes limitations

```
MATCH (s:Solution)-[:HAS_LIMITATION]->(l:Limitation)  
  
RETURN  
count(DISTINCT s) AS Number_of_Solutions,  
coalesce(l.description) AS Limitation  
ORDER BY Number_of_Solutions DESC, Limitation ASC
```